# Fundamentos do C#

Parte 1

Prof. Geovani

10/02/2026

# Marcos Importantes

| Data | Evento | Detalhes |
|---|---|---|
| 03/02 | ■ Aula 1 | Apresentação (Realizada) |
| 17/03 | ■ Entrega 1 | Proposta do Projeto |
| 05/05 | ■ AP (30%) | Avaliação Parcial |
| 26/05 | ■ Entrega 2 | Modelagem do BD |
| 02/06 | ■ Aula 16 | JWT - ÚLTIMA AULA |
| 09/06 | ■ AF (30%) | APRESENTAÇÃO FINAL |
| 16/06 | ■ AI (40%) | Avaliação Interdisciplinar |
| 23/06 | ■ AIS | Substitutiva (se aplicável) |
| 30/06 | ■ Aula 20 | Encerramento |

# Atenção ao nosso cronograma

| Aula | Data | Conteúdo Principal | Evento/Entrega |
|---|---|---|---|
| ~~1~~ | ~~03/02~~ | ~~Apresentação da Disciplina~~ | |
| 2 | 10/02 | Fundamentos C# - Parte 1 | - |
| - | 17/02 | CARNAVAL - Sem aula | Feriado |
| 3 | 24/02 | Fundamentos C# Parte 2 + Coleções | - |
| 4 | 03/03 | OOP - Parte 1 | - |
| 5 | 10/03 | OOP - Parte 2 | - |
| 6 | 17/03 | .NET Core e ASP.NET Core | Entrega Proposta de Projeto |
| 7 | 24/03 | SOLID + Design Patterns | - |
| 8 | 31/03 | Clean Architecture | - |
| - | 03-04/04 | Paixão de Cristo - Suspensão | Feriado |
| 9 | 07/04 | HTTP e REST | - |
| 10 | 14/04 | Construindo APIs REST | - |
| - | 21/04 | Tiradentes - Feriado | Feriado |
| 11 | 28/04 | Middleware e Filtros | - |
| 12 | 05/05 | AVALIAÇÃO PARCIAL (AP) | AP - 30% |
| 13 | 12/05 | PostgreSQL + EF Core | - |
| 14 | 19/05 | Repository Pattern + BD | - |
| 15 | 26/05 | Queries Avançadas | Entrega da Arquitetura de Software do projeto |
| 16 | **02/06** | **AUTENTICAÇÃO JWT** | **FIM CONTEÚDO** |
| - | 04-06/06 | Corpus Christi - Suspensão | Feriado |
| 17 | 09/06 | APRESENTAÇÃO FINAL (AF) | AF - 30% |
| 18 | 16/06 | AVALIAÇÃO INTERDISCIPLINAR (AI) | AI - 40% |
| 19 | 23/06 | Avaliação Substitutiva (AIS) | AIS |
| 20 | 30/06 | Encerramento + Revisão | Fim Semestre |

# Nossa Jornada

## 📚 BLOCO 1: FUNDAMENTOS

**Aulas 1-6:**
- C# e .NET Core
- Programação Orientada a Objetos
- Sintaxe e estruturas básicas
- Coleções e LINQ

## 🏗️ BLOCOS 2-5

**Aulas 7-20:**
- Arquitetura e SOLID
- REST APIs
- Banco de Dados
- Autenticação JWT

# Lista de Presença - Pt 1 (2 aulas)

https://lista-presenca.short.gy/tads-pt1

# Objetivos de Hoje

- Compreender a evolução do C# e .NET
- Dominar a sintaxe básica do C#
- Trabalhar com tipos de dados e operadores
- Implementar estruturas de controle e repetição
- Criar seu primeiro projeto Console
- Praticar com exercícios reais

# História e Evolução do C# e .NET

**Linha do Tempo:**

- 2002: C# 1.0 e .NET Framework (apenas Windows)
- 2015: C# 6.0 e surgimento do .NET Core 1.0
- 2020: .NET 5 - Unificação (Framework + Core)
- 2021: .NET 6 (LTS)
- 2022: .NET 7
- **2023: .NET 8 (LTS) ← Vamos usar este!**

# .NET Framework vs .NET Core vs .NET 5+

| Característica | .NET Framework | .NET Core | .NET 5+ |
|---|---|---|---|
| Plataforma | Apenas Windows | Multi-plataforma | Multi-plataforma |
| Open Source | Não | Sim | Sim |
| Performance | Boa | Excelente | Excelente |
| Versão Atual | 4.8 (final) | - | .NET 8 |
| Futuro | Sem novas versões | Descontinuado | Ativo e em evolução |

# Por que .NET Core é Multiplataforma?

- Reescrito do zero

Removidas dependências do Windows

- Modular e leve

Apenas bibliotecas necessárias

- Runtime CoreCLR

Compatível com Linux e macOS

- Desenvolvimento moderno

Contêineres Docker, cloud-native

💡 **Importante:**

Você pode desenvolver aplicações .NET no:

- • Windows
- • Linux
- • macOS

**E rodar em qualquer um deles!**

# Sintaxe Básica: Tipos de Dados Primitivos

| Tipo | Descrição | Exemplo | Tamanho |
|---|---|---|---|
| int | Número inteiro | 42, -10, 0 | 32 bits |
| double | Número decimal | 3.14, -0.5 | 64 bits |
| decimal | Decimal preciso | 19.99m | 128 bits |
| string | Texto | "Hello" | Variável |
| bool | Verdadeiro/Falso | true, false | 1 bit |
| char | Caractere único | 'A', 'z' | 16 bits |
| var | Tipo inferido | var x = 10; | - |

# Tipos Nullables

```
// Tipos normais NÃO podem ser null
int numero = null;  // ❌ ERRO!

// Tipos nullable PODEM ser null
int? numero = null;  // ✅ OK!
string? nome = null; // ✅ OK!

// Verificação
if (numero.HasValue)
{
    Console.WriteLine(numero.Value);
}
```

💡 **Quando usar?**

- • Valores opcionais
- • Dados de banco de dados
- • Parâmetros opcionais

# Variáveis e Constantes

**Declaração Explícita**

```
int idade = 25;
string nome = "João";
double preco = 99.90;
```

**Tipo Inferido (var)**

```
var idade = 25;         // int
var nome = "Maria";     // string
var ativo = true;       // bool
```

**Constantes**

```
const double PI = 3.14159;
const int MAX_USUARIOS = 100;
```

📌 **Boas Práticas:**

- • Use var quando o tipo for óbvio
- • Nomes descritivos: idade, não i
- • camelCase para variáveis
- • MAIÚSCULAS para constantes
- • const para valores fixos

# Operadores em C#

| Tipo | Operadores | Exemplo |
|---|---|---|
| Aritméticos | +, -, *, /, %, ++, -- | 5 + 3 = 8 |
| Relacionais | ==, !=, >, <, >=, <= | 5 > 3 → true |
| Lógicos | &&, \|\|, ! | true && false → false |
| Atribuição | =, +=, -=, *=, /= | x += 5 |

💡 **Operador ternário é um if compacto: (condição) ? valorSeVerdadeiro : valorSeFalso**

# Estruturas de Controle

**if / else if / else**

```
if (idade >= 18)
{
    Console.WriteLine("Maior de idade");
}
else if (idade >= 12)
{
    Console.WriteLine("Adolescente");
}
else
{
    Console.WriteLine("Criança");
}
```

**switch**

```
switch (diaSemana)
{
    case 1:
        Console.WriteLine("Segunda");
        break;
    case 2:
        Console.WriteLine("Terça");
        break;
    default:
        Console.WriteLine("Outro");
        break;
}
```

# Estruturas de Repetição (Loops)

```
for (int i = 0; i < 10; i++)
{
    Console.WriteLine(i);
}
```

```
int i = 0;
while (i < 10)
{
    Console.WriteLine(i++);
}
```

```
int i = 0;
do
{
    Console.WriteLine(i++);
}
while (i < 10);
```

```
string[] nomes = {"Ana", "João"};
foreach (string nome in nomes)
{
    Console.WriteLine(nome);
}
```

💡 **foreach é ideal para percorrer coleções sem precisar de índice**

# Controle de Loops: break, continue, return

| Comando | Função | Exemplo |
|---|---|---|
| break | Sai do loop imediatamente | if (x == 5) break; |
| continue | Pula para próxima iteração | if (x % 2 == 0) continue; |
| return | Sai da função | if (encontrou) return true; |

```
for (int i = 0; i < 10; i++)
{
    if (i == 5) break;         // Para quando i for 5
    if (i % 2 == 0) continue;  // Pula números pares
    Console.WriteLine(i);      // Imprime apenas ímpares até 5
}
```

# Hora de Praticar! 💻

Vamos criar nosso primeiro projeto Console

# Lista de Presença - Pt 2 (2 aulas)

https://lista-presenca.short.gy/tads-pt2

# Criando Primeiro Projeto Console

1. **Abra o Terminal / Prompt de Comando**

2. **Navegue até a pasta desejada:**
```
cd C:\MeusProjetos
```

3. **Crie o projeto:**
```
dotnet new console -n MeuPrimeiroProjeto
```

4. **Entre na pasta:**
```
cd MeuPrimeiroProjeto
```

5. **Abra no VS Code:**
```
code .
```

# Prática: Calculadora Básica

🎯 **Objetivo: Criar uma calculadora com 4 operações**

- 1. Solicite dois números ao usuário
- 2. Mostre menu com opções: +, -, *, /
- 3. Leia a operação escolhida
- 4. Execute e mostre o resultado

💡 **Dicas:**

- • Use Console.ReadLine() para ler entrada
- • Use int.Parse() ou Convert.ToInt32() para converter
- • Use switch para as operações

# Prática: Validação de Entrada do Usuário

```
// Lendo entrada
Console.Write("Digite sua idade: ");
string entrada = Console.ReadLine();

// Validando e convertendo
if (int.TryParse(entrada, out int idade))
{
    if (idade >= 18)
    {
        Console.WriteLine("Maior de idade");
    }
    else
    {
        Console.WriteLine("Menor de idade");
    }
}
else
{
    Console.WriteLine("Idade inválida!");
}
```

# Prática: Calculadora de IMC

🎯 **Objetivo: Calcular e classificar o IMC**

Fórmula: IMC = peso / (altura * altura)

**Classificação:**

- • < 18.5: Abaixo do peso
- • 18.5 - 24.9: Peso normal
- • 25 - 29.9: Sobrepeso
- • >= 30: Obesidade

✅ **Requisitos:**

- • Solicitar peso e altura
- • Calcular IMC
- • Mostrar classificação usando if/else if

# Prática: Tabuada

### 🎯 Exercício 1: Tabuada Simples

```
Console.Write("Digite um número: ");
int numero = int.Parse(Console.ReadLine());

for (int i = 1; i <= 10; i++)
{
    int resultado = numero * i;
    Console.WriteLine($"{numero} x {i} =
{resultado}");
}
```

### 🎯 Exercício 2: Desafio!

Crie um programa que:

- • Peça um número
- • Calcule o fatorial

**Exemplo:**
5! = 5 × 4 × 3 × 2 × 1 = 120

# Prática: Sequência de Fibonacci

🎯 **Objetivo: Gerar os primeiros N números de Fibonacci**

**O que é Fibonacci?**
Sequência onde cada número é a soma dos dois anteriores:
**0, 1, 1, 2, 3, 5, 8, 13, 21, 34...**

💡 **Dicas:**
- • Use duas variáveis para os dois últimos números
- • Use um loop for ou while
- • A cada iteração, calcule o próximo número
- • Atualize as variáveis para a próxima iteração

# 📝 Para Casa

### 📌 Exercício 1: Conversor de Unidades

Crie um programa que converta:

- • Celsius ↔ Fahrenheit
- • Quilômetros ↔ Milhas
- • Quilos ↔ Libras

### 📌 Exercício 2: Menu Interativo

Crie um menu com opções e use switch case

### 📌 Exercício 3: Números Primos

Verifique se um número é primo

# Próxima Aula

📚 Fundamentos de C# - Parte 2

- • Arrays e Coleções
- • Manipulação de Strings
- • Métodos e Funções
- • Tratamento de Exceções
- • Introdução ao LINQ

# Fim de Jogo

https://w.app/gil

Obrigado e até a próxima aula!